

Walchand College of Engineering, Sangli
Computer Science & Engineering
Third Year
Course: Computer Algorithm Lab

Assignment No. 7

Part: 2

Dynamic Programming

1. A hospital has **one operating room** and receives n surgery requests for the day. Each surgery i has a start time, end time, and a priority score (reflecting medical urgency and hospital revenue). Only non-overlapping surgeries can be performed in that room.

Task: Select a subset of non-overlapping surgeries that maximizes the total priority score.

Complexity twist: Two surgeries with different rooms can now run in parallel if the hospital has k operating rooms. Extend your solution to handle k rooms (surgeries assigned to any room, no two overlapping surgeries in the same room).

Input: n surgeries as (start, end, priority), number of rooms k .

Output: Maximum total priority achievable, and the schedule (which surgery $\rightarrow$ which room).

2. A factory has n production jobs. Each job i has a processing time, a deadline, and a penalty if the job is completed after its deadline (partial credit — the job still gets done, just late, at a cost). The factory has a single production line (jobs run one at a time, no preemption).

Task: Find the job order that minimizes the total penalty incurred across all jobs.

Input: n jobs as (processing_time, deadline, penalty).

Output: Minimum total penalty and the optimal job order.

3. A university must schedule n exams into time slots. Some pairs of exams **conflict** (share students) and cannot be scheduled in the same slot. Each exam also has a "difficulty weight," and the university wants to minimize the number of time slots used, while also ensuring that no student has two high-difficulty exams in adjacent slots (soft constraint, penalized).

Task: (a) Model exam conflicts as a graph and use graph coloring to find the minimum number of slots needed. (b) Given the minimum slot count, use DP over subsets (bitmask DP for small n) to also minimize the adjacent-difficulty penalty.

Input: n exams, conflict pairs, difficulty weights.

Output: Minimum number of slots, exam-to-slot assignment, total penalty.

4. A spell-checker application looks up n keywords, each with a known search frequency (probability of being searched). The company wants to build a Binary Search Tree over these keywords such that the **average search cost (number of comparisons)** is minimized, since frequently searched words should be closer to the root.

Task: Given n keys (sorted) with their search frequencies, construct the Optimal Binary Search Tree and compute its minimum expected search cost.

Input: n keys with frequencies $p_1 \dots p_n$ (and optionally dummy-key/failure probabilities $q_0 \dots q_n$ for unsuccessful searches).

Output: Minimum expected cost, and the structure of the optimal tree (root at each subrange).

5. An ISP wants to connect n rural towns with fiber-optic cable. The cost of laying cable directly between any two towns is known (based on terrain/distance). The ISP wants every town connected (directly or indirectly) to the network at **minimum total cost**, and wants to build the network incrementally starting from its main hub town.

Task: Using Prim's Algorithm, find the Minimum Spanning Tree (MST) starting from the hub town, and report the total cost and the cables to be laid.

Input: n towns, edge list with costs (or adjacency matrix), hub town.

Output: Total minimum cost, list of cables (edges) to lay.

6. A national power company must connect n cities to the power grid with minimum total transmission-line cost. Unlike the ISP scenario, the power company does not have a fixed starting hub — it wants to evaluate all possible connections and pick the globally cheapest set of connections that connects every city, while also respecting that some city pairs **cannot** be connected directly (e.g., geographic barriers — represented as missing edges).

Task: Using Kruskal's Algorithm (with Union-Find/Disjoint Set), find the MST. Additionally, detect and report if the graph is disconnected (i.e., no valid full spanning tree exists) and identify the disconnected components.

Input: n cities, list of possible connections with costs.

Output: Total minimum cost, selected edges, and (if applicable) disconnected components.

7. A delivery company has a drone that must visit n delivery points exactly once each and return to the base warehouse, minimizing total distance traveled. The drone has limited battery, so distances/costs between every pair of points are known and must be minimized exactly (n is small, $\leq 15-18$).

Task: Solve the Traveling Salesman Problem exactly using Bitmask Dynamic Programming (Held-Karp algorithm).

Input: n delivery points, distance matrix, base warehouse index.

Output: Minimum total route distance and the optimal visiting order.